xplainable Ensemble Learning Network Intrusion Detection

Abstract

Cyberattacks are growing in number and complexity, making it harder for traditional security systems to protect networks effectively. These systems depend on fixed rules that cannot detect new or unknown attacks. As a result, machine learning has become an important tool for building smarter Network Intrusion Detection Systems that can learn to identify threats from network traffic data.

Despite their promise, most machine learning-based detection systems face two critical problems. First, they cannot explain their decisions. When a system flags network traffic as an attack, analysts have no way of understanding why, which makes it difficult to trust or act on the alert. Second, these systems are trained once and never updated. As attackers change their methods over time, the model's performance quietly declines without anyone noticing — a problem known as concept drift.

This paper introduces a drift-aware explainable ensemble learning framework designed to solve both problems. The proposed system combines multiple machine learning models in a layered structure, where each model contributes its prediction and a final model learns how to combine them best. To make the system transparent, SHAP-based explanations are used to clearly show which features of the network traffic led to each detection decision. To keep the system reliable over time, an ADWIN-based drift detection mechanism watches for changes in network behavior and signals when the model needs to be updated. The framework is tested on the UNSW-NB15 dataset, confirming its ability to detect intrusions accurately, explain its decisions clearly, and adapt to changing network conditions.

Word Count: 240

Keywords

Network Intrusion Detection · Ensemble Learning · Explainable Artificial Intelligence · Concept Drift · ADWIN · SHAP

Research Gaps

Gap 1	Models Cannot Explain Their Decisions — Current systems detect attacks but cannot tell why. Analysts are left with no explanation, making it hard to trust or act on the output.
Gap 2	Models Stop Working Well Over Time — Systems trained on old data never update. As attackers change methods, accuracy drops silently with no built-in detection mechanism.
Gap 3	No System Combines Both Solutions — Explainability and drift detection are always studied separately. No existing framework brings both together in one unified IDS pipeline.

Gap 4	Single Models Used Instead of Combined Ones — Most IDS research relies on one classifier. Stacked ensemble learning, which gives better reliability, remains underexplored.

Research Objectives

Objective 1 — Build a Stacked Ensemble Model

Combine LightGBM, XGBoost, CatBoost, and Random Forest into a stacked architecture. A meta-model then learns the best way to combine their outputs for accurate intrusion detection.

Objective 2 — Add Explainability Using SHAP

Integrate a SHAP-based module that explains every detection decision, identifying which network features triggered the alert and which model contributed most.

Objective 3 — Detect Concept Drift Using ADWIN

Implement an ADWIN-based monitoring system that watches model performance during deployment and automatically raises a retraining alert when accuracy drops.

Objective 4 — Detect Zero-Day Attacks Using Isolation Forest

Add an Isolation Forest layer trained only on normal traffic that flags completely unknown attacks as zero-day threats, even when the classifier has never encountered them.

Objective 5 — Evaluate on Real-World Dataset

Test the entire framework on the UNSW-NB15 benchmark dataset and build a fully automated end-to-end Python pipeline covering data loading, training, SHAP analysis, and drift simulation.

Scope

This study focuses on building an intelligent Network Intrusion Detection System using machine learning techniques applied to the UNSW-NB15 benchmark dataset. The scope includes designing a stacked ensemble model for binary intrusion detection, integrating SHAP-based explainability, implementing Isolation Forest for zero-day attack identification, and applying ADWIN-based concept drift detection to monitor model performance over time. The study is limited to network traffic classification in a simulated environment and does not cover real-time deployment, hardware-level security, or encrypted traffic analysis.

Word Count: 100

Brief Overview of the Proposed Method

Stage 1 — Stacked Ensemble Detection

Four machine learning models — LightGBM, XGBoost, CatBoost, and Random Forest — are trained independently on network traffic data. Each model produces a probability score indicating whether the traffic is normal or an attack. A meta-level XGBoost classifier then combines these scores along with the original traffic features to make the final detection decision. This layered approach produces more reliable

results than any single model alone.

Stage 2 — Explainability and Zero-Day Detection

A SHAP-based module explains every prediction by identifying which features and which models influenced the decision most. Alongside this, an Isolation Forest is trained exclusively on normal traffic to detect completely new and unknown attacks that the classifier has never encountered before, flagging them as zero-day threats.

Stage 3 — Concept Drift Monitoring

An ADWIN-based drift detector continuously monitors the model's accuracy during deployment. When attack patterns change over time and performance degrades, the system automatically raises a retraining alert to keep the model up to date.

Word Count: 197

System Workflow

```
Network Traffic Data (UNSW-NB15) → Data Preprocessing (Clean | Encode | Scale) → Isolation Forest [Novelty 3: Zero-Day] + Base Models [LightGBM | XGBoost | CatBoost | RF] → Meta-Model XGBoost [Stacked Ensemble] → ■ Normal Traffic / ■ Attack Detected → SHAP Explain [Novelty 1] + ADWIN Monitor [Novelty 2]
```

Key Technologies

Language	Python 3.12
ML / Ensemble	LightGBM, XGBoost, CatBoost, scikit-learn (Random Forest, Isolation Forest, StandardScaler)
Explainability	SHAP (TreeSHAP / TreeExplainer)
Concept Drift	River library (ADWIN algorithm)
Data / Storage	Pandas, NumPy, CSV (UNSW-NB15 Dataset), Joblib (Model Saving)
Visualization	Matplotlib, Plotly
Dashboard	Streamlit
Version Control	Git / GitHub
Hardware Target	Apple Silicon CPU / NVIDIA GPU (optional, XGBoost GPU support)

Dataset Details

Property	Value
Name	UNSW-NB15
Total Records	257,673
Features	42 (after preprocessing)
Task	Binary — Normal vs Attack
Train Set	206,138 (80%)
Test Set	51,535 (20%)
Normal (test)	18,600
Attack (test)	32,935

Results

Base Model Performance

Model	Accuracy	Precision	Recall	F1-Score
LightGBM	95.02%	94.39%	94.91%	94.64%
XGBoost	95.04%	94.44%	94.88%	94.65%
CatBoost	94.77%	94.16%	94.57%	94.36%
Random Forest	93.84%	93.28%	93.40%	93.34%

Meta-Model — Final System

Accuracy	AUC-ROC	F1-Score	Precision	Recall
94.90%	99.20% ■	94.48%	94.44%	94.52%

Confusion Matrix

	Predicted Normal	Predicted Attack
Actual Normal	17,328 (TN) ■	1,272 (FP)
Actual Attack	1,356 (FN)	31,579 (TP) ■

SHAP Attribution Table — Novelty 1

Class	Top Contributor	LightGBM	XGBoost	CatBoost	RandomForest
Normal	LightGBM	2.3324	2.2675	0.4899	0.6309
Attack	LightGBM	2.9162	2.8152	0.3633	0.7355

Concept Drift — ADWIN — Novelty 2

Phase	Batches	Mean Accuracy	Std Dev

Pre-Drift	0 – 199	94.83%	±1.63%
Drifted	200 – 400	91.91%	±4.57%
Drop	—	–2.92%	—

Zero-Day Detection — Isolation Forest — Novelty 3

Precision	Recall	AUC-ROC	Training Samples (Normal)
91.77%	31.54%	79.43%	74,400

What is SHAP?

SHAP (SHapley Additive exPlanations) explains why a machine learning model made a specific prediction. After every detection, SHAP shows which network features pushed the prediction toward Attack and which pushed it toward Normal. This makes the system transparent and trustworthy for security analysts.

Example: duration = 0.001 sec → SHAP +2.91 (very short, suspicious — pushes toward Attack) | src_bytes = 999,999 → SHAP +1.83 (huge data — pushes toward Attack) | dst_port = 80 → SHAP –0.45 (normal web port — pushes toward Normal)

What is ADWIN?

ADWIN (ADaptive WINdowing) monitors a stream of accuracy values over time. It maintains a sliding window and checks whether recent average accuracy has significantly dropped compared to the earlier average. When it detects a drop, it raises a drift alert — signalling that the model needs to be retrained because attack patterns have changed.

Why is Meta-Model Slightly Less Than Individual Models?

The 0.14% difference between XGBoost (95.04%) and the Meta-Model (94.90%) is within the statistical margin of error — just 72 samples out of 51,535. The Meta-Model's true advantage is its AUC-ROC of 99.20% (higher than all individual models), its robustness when one model fails, and its better generalization through stacking which avoids overfitting.

How to Run

```
python Main.py --dataset UNSW_NB15 # Full pipeline (train + SHAP + drift + zero-day)
python show_results.py # View results in terminal + open plots
streamlit run dashboard.py # Open interactive web dashboard
```

Project Files

File	Purpose
Main.py	Runs the complete pipeline

data_preprocessing.py	Cleans and prepares data
base_models.py	Trains 4 base models
meta_model.py	Trains the meta-model
shap_explainability.py	SHAP explanations (Novelty 1)
concept_drift.py	ADWIN drift detection (Novelty 2)
anomaly_detector.py	Isolation Forest zero-day (Novelty 3)
evaluation.py	Generates all plots and metrics
dashboard.py	Streamlit web dashboard
show_results.py	Shows results in terminal